

Backend Engineering Challenge

Event-Driven Workflow Orchestration Engine

Objective

Design and implement a production-grade Event-Driven Workflow Engine. The system must accept JSON-based workflow definitions (Directed Acyclic Graphs), parse dependencies, and orchestrate the execution of tasks across distributed workers.

The engine must support parallel execution of independent branches, manage complex data passing between nodes, and handle asynchronous task lifecycles using a message broker.

An example payload for this task has been included as a separate attachment. Please use that payload when implementing your solution.

Important: We are not looking for authentication/authorization. Focus on the core orchestration logic, graph validation, and the event-driven architecture. **API endpoints must respond synchronously (acknowledging receipt), while workflow execution happens asynchronously in the background.**

You may use any tools, libraries, or AI assistance.

Technical Stack

- **Language:** Python 3.10+ (Type hinting required)
- **Database:** In-Memory (Redis) or PostgreSQL (Optional, for state persistence)
- **Message Broker:** Kafka / Redis
- **Framework:** FastAPI / Flask / Django
- **Containerization:** Docker & Docker Compose (all services must be containerized)

Core API Endpoints

Method	Endpoint	Description
POST	/workflow	Submit a new workflow JSON for execution. Returns execution_id.
POST	/workflow/trigger/:execution_id	Triggers the execution to start. Optional: submit node params.
GET	/workflows/:id	Retrieve the current status of the workflow (Pending, Running, Completed).
GET	/workflows/:id/results	Retrieve the final aggregated output of the workflow.

System Requirements

1. Workflow Parsing & Validation

- Before execution begins, the system must validate the integrity of the submitted JSON graph.
- Cycle Detection: The system must detect and reject workflows containing circular dependencies (e.g., $A \rightarrow B \rightarrow A$).
- Structure Validation: Ensure all nodes referenced in dependencies actually exist.
- DAG Construction: Convert the JSON into an internal graph representation ready for traversal.

2. Event-Driven Orchestration

- Implement an Orchestrator Service that manages the lifecycle of the workflow.
- Dependency Resolution: The Orchestrator must determine which nodes are eligible to run (nodes with 0 dependencies or nodes where all parents are COMPLETED).
- Parallelism: Independent nodes must be dispatched simultaneously.
- State Management: The system must track the state of every node (**PENDING, RUNNING, COMPLETED, FAILED**).

3. Data Passing & Template Resolution

The engine must support dynamic data injection between nodes.

- Outputs: Every node produces a JSON output upon completion.

- Inputs: Subsequent nodes can reference these outputs using a specific syntax (e.g., {{ node_name.output_key }}).
- Resolution Logic: Before dispatching a task to a worker, the Orchestrator must resolve these templates into actual values.

4. Worker Implementation & Mocking

- Implement Worker Services that consume tasks from the message broker. Do not call real external APIs.
- External Service Node: Mock HTTP calls by sleeping for 1-2 seconds and returning a dummy JSON object.
- LLM Service Node: Mock an AI generation process. Accept a prompt (with resolved variables) and return a mock string response.
- Idempotency: Workers should handle receiving the same task message twice without executing logic twice.

Core Functionality Scenarios

Scenario A (Linear): $A \rightarrow B \rightarrow C$. C must receive data from B (or A).

Scenario B (Fan-Out/Fan-In): A triggers B and C (in parallel). D waits for both B and C to finish. D aggregates results from both.

Scenario C (Race Conditions): Ensure that if B and C finish at the exact same millisecond, the Orchestrator correctly triggers D exactly once.

Deliverables

- **Source Code:** Production-grade Python code submitted in a Git repository.
- **docker-compose.yml:** Single command to start API, Orchestrator, Message Broker, and Worker(s).
- **README.md:** Setup instructions, architecture overview, and how to trigger the test workflows.
- **DESIGN.md:** Document your key design decisions (detecting readiness, handling fan-in, trade-offs).
- **Tests:** Tests covering the important edge cases.

Good luck! We look forward to reviewing your solution. 🚀